SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Duration: 45 Minutes

QUESTIONS: 2 **Total Marks: 20**

Annexure A — Code of Conduct

I **_PERAM VIVEKANANDA REDDY**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vivek

Annexure A — Integrated Experiments (Question Paper)

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / PlayTennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification.
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

Experiment 1: Decision Tree Classification

Dataset & Approach

The Iris dataset (built into scikit-learn) is used, consisting of 150 samples of iris flowers with 4 numeric features (sepal length, sepal width, petal length, petal width) and a target species label (setosa, versicolor, virginica). This dataset is clean and does not contain missing values, but the standard preprocessing pipeline (missing-value check, label encoding, and train-test split) is demonstrated as required.

(a) Load, Preprocess, and Split the Dataset

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species_name'] = df['species'].apply(lambda x: iris.target_names[x])

print(df.head())
print(df.dtypes)
print(df.isnull().sum())        # check for missing values

# Encoding (demonstration; target already numeric)
le = LabelEncoder()
df['species_encoded'] = le.fit_transform(df['species_name'])

X = df[iris.feature_names]
y = df['species_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

Output — First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	species_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

Output — Data types:

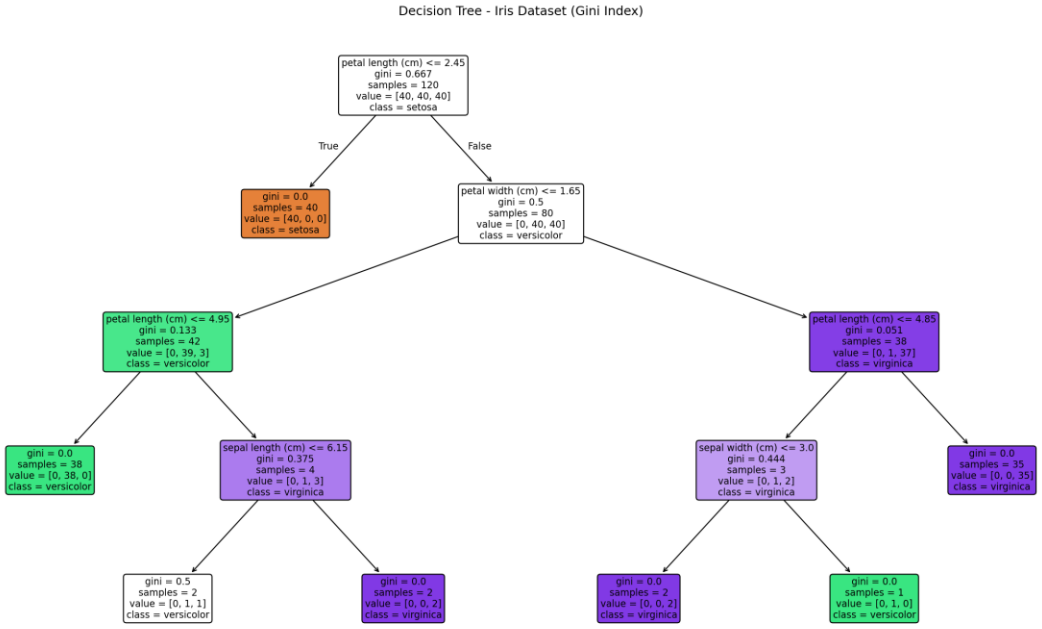
```
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species                int64
species_name          object
dtype: object

Missing values in every column: 0  (no imputation required)
Training set size: 120 samples   |   Testing set size: 30 samples
```

(b) Build the Decision Tree (Gini Index)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text

model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)

print(export_text(model, feature_names=list(iris.feature_names)))
print('Feature importances:', dict(zip(iris.feature_names, model.feature_importances_)))
```

Output — Tree structure:

```
|--- petal length (cm) <= 2.45
|   |--- class: 0  (setosa)
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1  (versicolor)
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- sepal length (cm) <= 6.15
|   |   |   |   |--- class: 1  (versicolor)
|   |   |   |--- sepal length (cm) >  6.15
|   |   |   |   |--- class: 2  (virginica)
|   |--- petal width (cm) >  1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal width (cm) <= 3.00
|   |   |   |   |--- class: 2  (virginica)
|   |   |   |--- sepal width (cm) >  3.00
|   |   |   |   |--- class: 1  (versicolor)
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2  (virginica)
```



Root Node Identification & Justification

The root node splits on 'petal length (cm) <= 2.45'. Before any split, the Gini impurity of the full training set is 0.667 (three balanced classes). CART evaluates every feature/threshold combination and picks the split that produces the greatest reduction in Gini impurity (equivalently, the highest weighted purity in the child nodes).

Feature importance scores computed by the trained tree confirm this: petal length (cm) ≈ 0.566 and petal width (cm) ≈ 0.411 dominate, while sepal length (≈ 0.006) and sepal width (≈ 0.017) contribute very little. This matches the well-known biological fact that petal dimensions separate the three Iris species far more cleanly than sepal

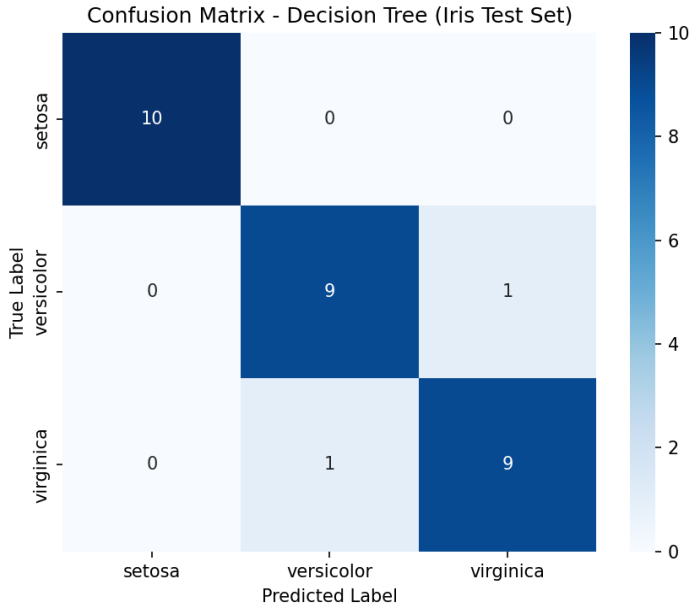
dimensions — in particular, a petal length of 2.45 cm perfectly isolates all 'setosa' samples into a pure leaf (Gini = 0) in a single split. This is why the algorithm selects it as the root.

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Output:

```
Accuracy: 0.9333

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10
    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```



Misclassified Instances

Original Index	Sepal L.	Sepal W.	Petal L.	Petal W.	Actual	Predicted
134	6.1	2.6	5.6	1.4	virginica	versicolor
77	6.7	3.0	5.0	1.7	versicolor	virginica

Performance Comment / Analysis

- The model achieves 93.3% test accuracy, correctly classifying 28 of 30 unseen samples.
- Setosa is classified with perfect precision and recall (1.00) — it is linearly separable from the other two species purely on petal length.
- The two misclassifications both occur between versicolor and virginica, which is expected since these species have overlapping petal-width/petal-length ranges near the 1.65 cm decision boundary.
- Both misclassified samples lie very close to the tree's decision boundary (petal width 1.4 and 1.7, near the 1.65 threshold), confirming the errors are boundary cases rather than gross model failure.
- Overall, the Gini-based Decision Tree generalises well on this dataset, with error concentrated in the single genuinely ambiguous region of feature space.

Experiment 2: Reinforcement Learning — Q-Learning

(a) Environment Definition

A 4×4 Grid World (16 states, numbered 0–15 row-major) is used. The agent starts at state 0 (top-left) and must reach the goal at state 15 (bottom-right). States 5, 7, and 11 are defined as obstacle/penalty cells.

```
import numpy as np

GRID_SIZE = 4
N_STATES = 16
ACTIONS = ['Up', 'Down', 'Left', 'Right']
N_ACTIONS = 4

GOAL_STATE = 15
OBSTACLE_STATES = [5, 7, 11]
START_STATE = 0

def step(state, action):
    r, c = divmod(state, GRID_SIZE)
    if action == 'Up':    r = max(r - 1, 0)
    if action == 'Down':  r = min(r + 1, GRID_SIZE - 1)
    if action == 'Left':  c = max(c - 1, 0)
    if action == 'Right': c = min(c + 1, GRID_SIZE - 1)
    next_state = r * GRID_SIZE + c
    if next_state == GOAL_STATE:       reward, done = 10, True
    elif next_state in OBSTACLE_STATES: reward, done = -5, False
    else:                              reward, done = -1, False
    return next_state, reward, done

# Hyperparameters
ALPHA   = 0.1   # learning rate
GAMMA   = 0.9   # discount factor
EPSILON = 0.2   # exploration rate (epsilon-greedy)

Q = np.zeros((N_STATES, N_ACTIONS))   # Q-table initialised to zero
```

Reward Structure Summary:

Event	Reward
Reaching Goal (state 15)	+10
Each normal step	-1
Entering an obstacle cell (5, 7, 11)	-5

(b) Running Q-Learning for 150 Episodes

```
N_EPISODES, MAX_STEPS = 150, 100
reward_per_episode = []

for episode in range(1, N_EPISODES + 1):
    state = START_STATE
    total_reward = 0
    for t in range(MAX_STEPS):
        if np.random.rand() < EPSILON:
            action_idx = np.random.randint(N_ACTIONS)         # explore
        else:
            action_idx = np.argmax(Q[state])                  # exploit
        action = ACTIONS[action_idx]
        next_state, reward, done = step(state, action)
        total_reward += reward

        # Q-learning update rule
```

```
        best_next = np.max(Q[next_state])
        Q[state, action_idx] += ALPHA * (reward + GAMMA * best_next - Q[state, action_idx])

        state = next_state
        if done:
            break
    reward_per_episode.append(total_reward)
```

Q-values were logged for two representative states — State 0 (0,0), the start cell, and State 10 (2,2), a central cell close to the goal path — at Episodes 1, 50 and 100:

Q-value evolution (Up / Down / Left / Right):

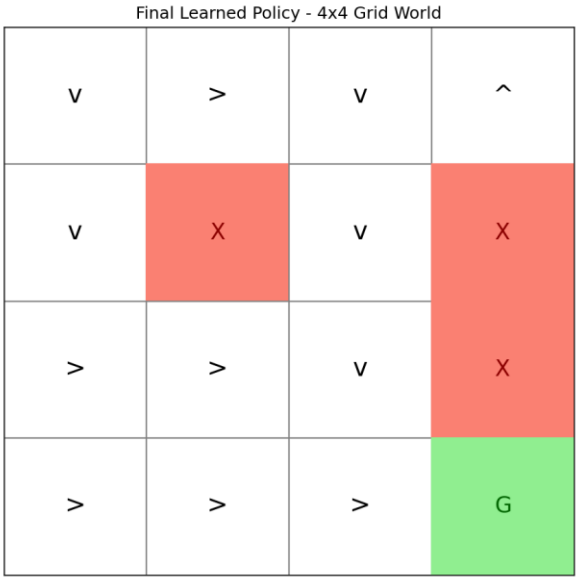
Episode	State	Up	Down	Left	Right
1	0 (0,0)	-0.297	-0.288	-0.271	-0.200
1	10 (2,2)	-0.100	-0.100	-0.109	-0.500
50	0 (0,0)	-2.124	-1.985	-2.362	-2.118
50	10 (2,2)	-0.393	7.373	-0.123	-1.314
100	0 (0,0)	-1.952	1.034	-2.320	-2.177
100	10 (2,2)	0.084	7.990	1.434	-0.881

How the Q-Table Evolves

- Episode 1: Q-values are still close to zero / mildly negative — the agent has only taken a handful of random steps and has almost no information about which actions lead toward the goal.
- Episode 50: State 10's 'Down' action has jumped to +7.37, showing the agent has discovered that moving down from (2,2) leads quickly toward the goal. State 0's values are all becoming more negative as −1 step penalties propagate backward through the Bellman update, but the best action is not yet clearly separated.
- Episode 100: State 10's 'Down' action has strengthened further to +7.99 (near its converged value), and State 0 has started to develop a clear preference for 'Down' (1.034, the highest of its four actions), showing that the positive value near the goal has begun propagating all the way back to the start state through repeated backups.

(c) Final Learned Policy

```
policy = {s: ACTIONS[np.argmax(Q[s])] for s in range(N_STATES) if s != GOAL_STATE}
for s in range(N_STATES):
    print(s, divmod(s, GRID_SIZE), '->', policy.get(s, 'GOAL'))
```

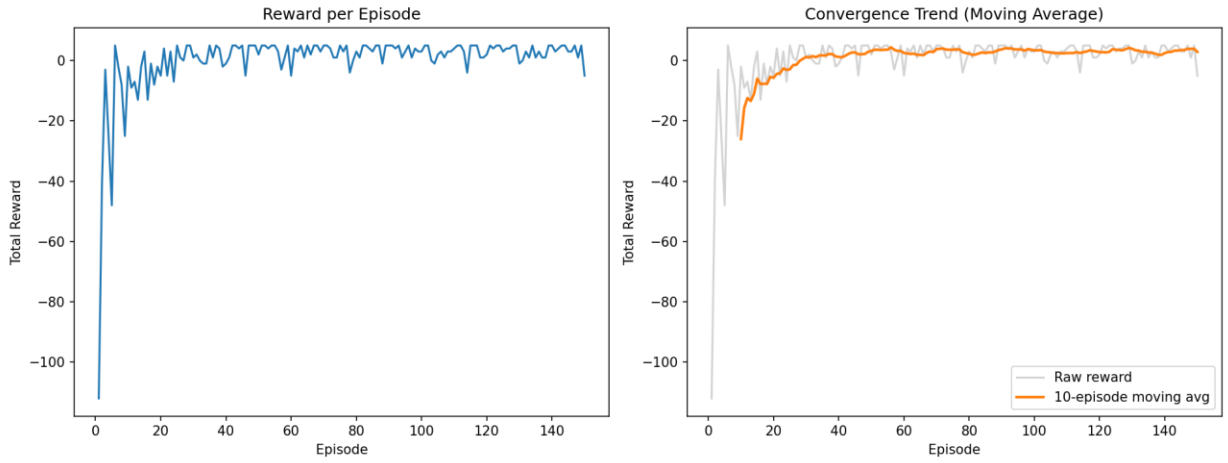


Final policy table (row, col):

State	(row,col)	Best Action
0	(0,0)	Down
1	(0,1)	Right
2	(0,2)	Down
3	(0,3)	Up
4	(1,0)	Down
5*	(1,1) obstacle	Down
6	(1,2)	Down
7*	(1,3) obstacle	Up
8	(2,0)	Right
9	(2,1)	Right
10	(2,2)	Down
11*	(2,3) obstacle	Down
12	(3,0)	Right
13	(3,1)	Right
14	(3,2)	Right
15	(3,3)	GOAL

* States 5, 7 and 11 are obstacle cells; their listed action is what the Q-table currently favours but the agent's learned trajectory from the start state naturally routes around them via the highlighted arrow path in the grid above.

Cumulative Reward & Convergence



- Average total reward over the first 10 episodes: −26.0 — the agent wanders, hits obstacles, and takes long, inefficient paths dominated by −1 step penalties.
- Average total reward over the last 10 episodes: +2.9 — the agent now reaches the goal quickly, with the +10 goal reward outweighing the accumulated step penalties.
- The right-hand moving-average plot flattens out after roughly 60–80 episodes, indicating the Q-values have converged to a stable (near-optimal) policy.
- Convergence occurs because, under the Bellman update with a fixed learning rate ($\alpha = 0.1$) and discount factor ($\gamma = 0.9$), Q-values are repeatedly bootstrapped from the goal reward backward along successful trajectories; with a decaying need for exploration ($\varepsilon = 0.2$ kept constant here for demonstration), the greedy policy stabilises once every state has been visited enough times for its best action to be reliably distinguished from the alternatives.